

TUGAS MANDIRI MODUL 13
DASAR KECERDASAN ARTIFISIAL



Universitas
Telkom

Dipersiapkan Oleh :
Rovino Ramadhani – 103072400031

UNIVERSITAS TELKOM SURABAYA
PROGRAM STUDI INFORMATIKA
2026

Code

<https://github.com/rovinor4/Praktikum-Dasar-Kecerdasan-Artifisial/tree/main/Tugas%20Mandiri%20-%20Modul%202013%20FUZZY%20LOGIC>

Penjelasan Dan Output Soal 1

Import library dan pendefinisian variabel simbolik

```
In [55]: import sympy as sp  
  
x = sp.symbols("x")
```

Library SymPy, yaitu library Python yang sering dipakai untuk perhitungan matematika simbolik.

Variabel x dibuat sebagai variabel simbolik. Variabel ini nantinya digunakan sebagai dasar dalam membentuk fungsi keanggotaan fuzzy. Karena fungsi keanggotaan seperti suhu dingin, suhu sejuk, kelembaban lembab, dan kecepatan kipas dibuat dalam bentuk fungsi terhadap nilai x, maka x perlu didefinisikan terlebih dahulu.

Fungsi trimf

```
In [56]: def trimf(a, b, c):  
    return sp.Piecewise(  
        (0, x <= a),  
        ((x - a) / (b - a), x <= b),  
        ((c - x) / (c - b), x < c),  
        (0, True)  
    )
```

Fungsi trimf, yaitu singkatan dari triangular membership function atau fungsi keanggotaan berbentuk segitiga.

Fungsi ini menerima tiga parameter, yaitu a, b, dan c. Parameter a adalah batas awal, b adalah titik puncak, dan c adalah batas akhir. Nilai keanggotaan akan bernilai 0 sebelum titik a, naik secara bertahap dari a ke b, mencapai nilai maksimum 1 di sekitar titik b, lalu turun kembali menuju 0 sampai titik c.

Fungsi ini digunakan untuk kategori fuzzy yang memiliki bentuk naik lalu turun, misalnya kategori "Sejuk", "Normal", "Pelan", "Sedang", dan "Cepat".

Fungsi trapmf

```
In [57]: def trapmf(a, b, c, d):  
    if a == b:  
        return sp.Piecewise(  
            (0, x < a),  
            (1, x <= c),  
            ((d - x) / (d - c), x < d),  
            (0, True)  
        )  
    if c == d:  
        return sp.Piecewise(  
            (0, x <= a),  
            ((x - a) / (b - a), x < b),  
            (1, x <= d),  
            (0, True)  
        )  
    return sp.Piecewise(  
        (0, x <= a),  
        ((x - a) / (b - a), x < b),  
        (1, x <= c),  
        ((d - x) / (d - c), x < d),  
        (0, True)  
    )
```

Fungsi trapmf, yaitu trapezoidal membership function atau fungsi keanggotaan berbentuk trapesium.

Fungsi ini menerima empat parameter, yaitu a, b, c, dan d. Secara umum, fungsi ini memiliki empat bagian. Bagian pertama bernilai 0, bagian kedua naik secara bertahap, bagian ketiga bernilai penuh 1, dan bagian keempat turun kembali ke 0.

Namun, fungsi ini juga menangani dua kondisi khusus.

Kondisi pertama adalah ketika $a == b$. Artinya, fungsi langsung berada pada nilai maksimum sejak awal batas. Ini cocok untuk kategori seperti "Dingin", "Kering", atau "Mati", karena kategori tersebut dimulai dari nilai paling rendah dan langsung memiliki derajat keanggotaan penuh.

Kondisi kedua adalah ketika $c == d$. Artinya, fungsi tetap bernilai maksimum hingga batas akhir. Ini cocok untuk kategori seperti "Panas", "Lembab", atau "Maksimal", karena kategori tersebut berada di ujung kanan rentang nilai.

Fungsi degree

```
In [58]: def degree(expr, nilai):  
    return float(expr.subs(x, nilai))
```

Fungsi degree, yaitu fungsi untuk menghitung nilai derajat keanggotaan dari

sebuah fungsi fuzzy.

Parameter `expr` berisi fungsi keanggotaan fuzzy, sedangkan nilai adalah nilai input yang ingin diuji. Program mengganti variabel simbolik `x` dengan nilai aktual, lalu hasilnya dikonversi menjadi angka desimal bertipe float.

Contohnya, jika input suhu adalah 28 derajat, maka fungsi ini dapat digunakan untuk mencari seberapa besar derajat keanggotaan suhu tersebut terhadap kategori "Sejuk" atau "Panas".

Fungsi Input Range

```
In [59]: def input_range(teks, minimum, maksimum):  
    while True:  
        try:  
            nilai = float(input(teks))  
            if minimum <= nilai <= maksimum:  
                return nilai  
            print(f"Input harus dalam range {minimum}-{maksimum}.")  
        except ValueError:  
            print("Input harus berupa angka.")
```

Fungsi `input_range`, yaitu fungsi untuk menerima input angka dari pengguna sekaligus memastikan nilainya berada dalam rentang yang diperbolehkan.

Fungsi ini berjalan dalam perulangan sampai pengguna memasukkan nilai yang valid. Jika input berupa angka dan berada di antara batas minimum serta maksimum, maka nilai tersebut dikembalikan. Jika nilai berada di luar rentang, program menampilkan pesan bahwa input harus sesuai range. Jika input bukan angka, program menampilkan pesan bahwa input harus berupa angka.

Fungsi ini digunakan agar program tidak memproses data yang tidak logis. Misalnya, suhu ruangan dibatasi antara 0 sampai 50 derajat, sedangkan kelembaban dibatasi antara 0 sampai 100 persen.

Fungsi centroid, yaitu fungsi untuk melakukan proses defuzzifikasi.

```
In [60]: def centroid(output_mf, aturan, start, end, step):  
    total_titik = int((end - start) / step) + 1  
    pembilang = 0.0  
    penyebut = 0.0  
  
    for i in range(total_titik):  
        nilai = start + i * step  
        mu = max(min(w, degree(output_mf[label], nilai)) for label,  
        pembilang += nilai * mu  
        penyebut += mu
```

```
return pembilang / penyebut if penyebut != 0 else 0.0
```

Dalam sistem fuzzy, hasil inferensi masih berupa derajat keanggotaan pada beberapa kategori output. Agar hasil akhirnya menjadi satu angka pasti, digunakan metode defuzzifikasi. Pada program ini, metode yang digunakan adalah metode centroid.

Fungsi ini menerima beberapa parameter. `output_mf` berisi fungsi keanggotaan output, aturan berisi hasil aturan fuzzy beserta kekuatan aktifnya, `start` dan `end` adalah batas rentang output, sedangkan `step` adalah jarak antar titik perhitungan.

Program menghitung banyak titik dari rentang output. Dalam kasus ini, rentangnya 0 sampai 100 dengan step 0.1, sehingga jumlah titiknya adalah 1001 titik.

Pada setiap titik, program menghitung nilai keanggotaan output berdasarkan aturan yang aktif. Nilai `min(w, degree(...))` digunakan untuk membatasi fungsi output sesuai firing strength aturan. Kemudian `max(...)` digunakan untuk menggabungkan semua aturan fuzzy yang menghasilkan output.

Variabel `pembilang` menghitung jumlah nilai output dikalikan derajat keanggotaannya. Variabel `penyebut` menghitung total derajat keanggotaan. Hasil akhir centroid diperoleh dari pembilang dibagi penyebut.

Jika penyebut bernilai 0, artinya tidak ada aturan yang aktif, maka fungsi mengembalikan nilai 0.

```
In [61]: def interpretasi(nilai):  
        if nilai <= 25:  
            return "Mati"  
        if nilai <= 45:  
            return "Pelan"  
        if nilai <= 65:  
            return "Sedang"  
        if nilai <= 85:  
            return "Cepat"  
        return "Maksimal"
```

Blok ini membuat fungsi `interpretasi`, yaitu fungsi untuk mengubah nilai numerik kecepatan kipas menjadi kategori bahasa.

Jika nilai hasil defuzzifikasi kurang dari atau sama dengan 25, maka interpretasinya "Mati". Jika nilainya sampai 45, maka "Pelan". Jika sampai 65, maka "Sedang". Jika sampai 85, maka "Cepat". Jika lebih dari 85, maka "Maksimal".

Input

```
In [62]: suhu_input = input_range("Masukkan suhu ruangan (0-50): ", 0, 50)
kelembaban_input = input_range("Masukkan kelembaban ruangan (0-100)
```

memasukkan dua nilai, yaitu suhu ruangan dan kelembaban ruangan.

Input suhu dibatasi dari 0 sampai 50, sedangkan input kelembaban dibatasi dari 0 sampai 100. Kedua input menggunakan fungsi `input_range`, sehingga program akan memastikan bahwa nilai yang dimasukkan valid.

Fungsi keanggotaan untuk tiga variabel fuzzy, yaitu suhu, kelembaban, dan kecepatan kipas.

```
In [63]: suhu_mf = {
    "Dingin": trapmf(0, 0, 10, 20),
    "Sejuk": trimf(15, 22, 30),
    "Panas": trapmf(25, 35, 50, 50)
}

kelembaban_mf = {
    "Kering": trapmf(0, 0, 30, 50),
    "Normal": trimf(35, 50, 65),
    "Lembab": trapmf(55, 70, 100, 100)
}

kipas_mf = {
    "Mati": trapmf(0, 0, 10, 25),
    "Pelan": trimf(15, 35, 55),
    "Sedang": trimf(40, 55, 70),
    "Cepat": trimf(55, 75, 90),
    "Maksimal": trapmf(80, 90, 100, 100)
}

suhu = {k: degree(v, suhu_input) for k, v in suhu_mf.items()}
kelembaban = {k: degree(v, kelembaban_input) for k, v in kelembaban_mf.items()}
```

Pada variabel suhu, terdapat tiga kategori. "Dingin" menggunakan fungsi trapesium, "Sejuk" menggunakan fungsi segitiga, dan "Panas" menggunakan fungsi trapesium. Ini menggambarkan bahwa suhu rendah, sedang, dan tinggi memiliki bentuk keanggotaan yang berbeda.

Pada variabel kelembaban, terdapat tiga kategori. "Kering" menggunakan fungsi trapesium, "Normal" menggunakan fungsi segitiga, dan "Lembab" menggunakan fungsi trapesium.

Pada variabel output kecepatan kipas, terdapat lima kategori. Kategori tersebut adalah "Mati", "Pelan", "Sedang", "Cepat", dan "Maksimal". Kategori ekstrem seperti "Mati" dan "Maksimal" menggunakan trapesium, sedangkan kategori tengah menggunakan segitiga.

Setelah fungsi keanggotaan didefinisikan, program menghitung derajat

keanggotaan suhu input terhadap semua kategori suhu. Program juga menghitung derajat keanggotaan kelembaban input terhadap semua kategori kelembaban.

Mendefinisikan sembilan aturan fuzzy.

```
In [64]: rules = [
    ("R1", "Dingin", "Kering", "Mati"),
    ("R2", "Dingin", "Normal", "Mati"),
    ("R3", "Dingin", "Lembab", "Pelan"),
    ("R4", "Sejuk", "Kering", "Pelan"),
    ("R5", "Sejuk", "Normal", "Sedang"),
    ("R6", "Sejuk", "Lembab", "Cepat"),
    ("R7", "Panas", "Kering", "Sedang"),
    ("R8", "Panas", "Normal", "Cepat"),
    ("R9", "Panas", "Lembab", "Maksimal")
]
```

Setiap aturan terdiri dari kode aturan, kategori suhu, kategori kelembaban, dan kategori output kecepatan kipas. Aturan ini menggunakan pola logika sederhana: jika suhu berada pada kategori tertentu dan kelembaban berada pada kategori tertentu, maka kipas berada pada kategori tertentu.

Contohnya, jika suhu "Dingin" dan kelembaban "Kering", maka kipas "Mati". Jika suhu "Panas" dan kelembaban "Lembab", maka kipas "Maksimal".

Karena suhu memiliki tiga kategori dan kelembaban juga memiliki tiga kategori, maka total kombinasi aturannya adalah 3×3 , yaitu sembilan aturan.

Perhitungan firing strength, centroid, dan aturan aktif

```
In [65]: firing = []
for code, s, k, output in rules:
    w = min(suhu[s], kelembaban[k])
    firing.append((code, s, k, output, w))

hasil = centroid(kipas_mf, [(output, w) for _, _, _, output, w in firing])
aktif = [code for code, _, _, _, w in firing if w > 0]
```

Program membuat list kosong bernama firing. Kemudian setiap aturan diproses satu per satu. Untuk setiap aturan, program mengambil derajat keanggotaan suhu dan derajat keanggotaan kelembaban sesuai kategori pada aturan tersebut.

Nilai firing strength dihitung menggunakan operasi minimum. Hal ini sesuai dengan operator AND dalam logika fuzzy, yaitu mengambil nilai paling kecil dari dua derajat keanggotaan.

Setiap hasil aturan kemudian disimpan ke dalam list firing, lengkap dengan

kode aturan, kategori suhu, kategori kelembaban, output, dan nilai firing strength.

Setelah itu, program menghitung hasil akhir menggunakan fungsi centroid. Output fuzzy yang sudah dipotong berdasarkan firing strength diproses menjadi satu nilai tegas dalam rentang 0 sampai 100.

Program juga membuat daftar aktif, yaitu daftar aturan yang memiliki firing strength lebih dari 0. Aturan dengan nilai lebih dari 0 dianggap aktif karena ikut memengaruhi hasil akhir.

Output

```
In [66]: print("\n-----")
print("INPUT:")
print("-----")
print(f"Suhu: {suhu_input:.2f} °C")
print(f"Kelembaban: {kelembaban_input:.2f} %")
```

```
-----
INPUT:
-----
```

```
Suhu: 28.00 °C
Kelembaban: 65.00 %
```

```
In [67]: print("\n-----")
print("FUZZIFIKASI")
print("-----")
print("Suhu:")
print(f"Dingin: {suhu['Dingin']:.6f}")
print(f"Sejuk: {suhu['Sejuk']:.6f}")
print(f"Panas: {suhu['Panas']:.6f}")
```

```
-----
FUZZIFIKASI
-----
```

```
Suhu:
Dingin: 0.000000
Sejuk: 0.250000
Panas: 0.300000
```

```
In [68]: print("\n-----")
print("Kelembaban:")
print("-----")
print(f"Kering: {kelembaban['Kering']:.6f}")
print(f"Normal: {kelembaban['Normal']:.6f}")
print(f"Lembab: {kelembaban['Lembab']:.6f}")
```

```
-----
Kelembaban:
-----
```

```
Kering: 0.000000
Normal: 0.000000
Lembab: 0.666667
```



```
In [69]: print("\n-----")
print("FIRING STRENGTH (9 Aturan)")
print("-----")
for kode, s, k, output, w in firing:
    print(f"{kode} ({s} AND {k} -> {output}): {w:.6f}")
```

```
-----
FIRING STRENGTH (9 Aturan)
-----
```

```
R1 (Dingin AND Kering -> Mati): 0.000000
R2 (Dingin AND Normal -> Mati): 0.000000
R3 (Dingin AND Lembab -> Pelan): 0.000000
R4 (Sejuk AND Kering -> Pelan): 0.000000
R5 (Sejuk AND Normal -> Sedang): 0.000000
R6 (Sejuk AND Lembab -> Cepat): 0.250000
R7 (Panas AND Kering -> Sedang): 0.000000
R8 (Panas AND Normal -> Cepat): 0.000000
R9 (Panas AND Lembab -> Maksimal): 0.300000
```

```
In [70]: print("\n-----")
print("DEFUZZIFIKASI")
print("-----")
print("Metode: Centroid (1001 titik)")
print(f"Kecepatan Kipas: {hasil:.2f} %")
print(f"Interpretasi: {interpretasi(hasil)}")
```

```
-----
DEFUZZIFIKASI
-----
```

```
Metode: Centroid (1001 titik)
Kecepatan Kipas: 79.70 %
Interpretasi: Cepat
```

```
In [71]: print("\n-----")
print("TOLAK UKUR VALIDASI")
print("-----")
print(f"μ_Sejuk({suhu_input:.2f}) = {suhu['Sejuk']:.6f}")
print(f"μ_Panas({suhu_input:.2f}) = {suhu['Panas']:.6f}")
print(f"μ_Lembab({kelembaban_input:.2f}) = {kelembaban['Lembab']:.6f}")
print(f"Aturan aktif: {'', '.join(aktif) if aktif else 'Tidak ada'}")
print(f"Centroid berada di kisaran 75-85%: {'Ya' if 75 <= hasil <= 85 else 'Tidak'}")
print(f"Interpretasi akhir: {interpretasi(hasil)}")
```

```
-----
TOLAK UKUR VALIDASI
-----
```

```
μ_Sejuk(28.00) = 0.250000
μ_Panas(28.00) = 0.300000
μ_Lembab(65.00) = 0.666667
Aturan aktif: R6, R9
Centroid berada di kisaran 75-85%: Ya
Interpretasi akhir: Cepat
```

Penjelasan Dan Output Soal 2

Import SymPy dan variabel x

```
In [37]: import sympy as sp  
  
x = sp.symbols("x")
```

Bagian ini menyiapkan library SymPy untuk membuat fungsi matematika berbasis simbol. Variabel x digunakan sebagai variabel utama dalam semua fungsi keanggotaan fuzzy, sehingga nilai input bisa disubstitusikan ke dalam fungsi tersebut.

Fungsi trimf

```
In [38]: def trimf(a, b, c):  
    return sp.Piecewise(  
        (0, x <= a),  
        ((x - a) / (b - a), x <= b),  
        ((c - x) / (c - b), x < c),  
        (0, True)  
    )
```

Fungsi trimf digunakan untuk membuat fungsi keanggotaan berbentuk segitiga. Fungsi ini cocok untuk kategori yang memiliki satu titik puncak di tengah, seperti "Biasa".

Nilai keanggotaan akan bernilai 0 sebelum batas awal, naik perlahan sampai titik tengah, lalu turun lagi sampai batas akhir. Dalam program ini, fungsi tersebut dipakai untuk menggambarkan kualitas makanan atau pelayanan yang berada di kategori sedang.

Fungsi trapmf

```
In [39]: def trapmf(a, b, c, d):  
    if a == b:  
        return sp.Piecewise(  
            (0, x < a),  
            (1, x <= c),  
            ((d - x) / (d - c), x < d),  
            (0, True)  
        )  
  
    if c == d:  
        return sp.Piecewise(  
            (0, x <= a),  
            ((x - a) / (b - a), x < b),
```

```

        (1, x <= d),
        (0, True)
    )

    return sp.Piecewise(
        (0, x <= a),
        ((x - a) / (b - a), x < b),
        (1, x <= c),
        ((d - x) / (d - c), x < d),
        (0, True)
    )

```

Fungsi trapmf digunakan untuk membuat fungsi keanggotaan berbentuk trapesium. Fungsi ini cocok untuk kategori yang berada di ujung rendah atau ujung tinggi, seperti "Buruk", "Enak", dan "Baik".

Jika batas awal sama, fungsi langsung bernilai penuh di bagian awal. Jika batas akhir sama, fungsi bisa tetap bernilai penuh sampai ujung akhir. Hal ini membuat fungsi trapesium lebih fleksibel untuk menggambarkan kategori ekstrem.

Fungsi degree

```

In [40]: def degree(expr, nilai):
          return float(expr.subs(x, nilai))

```

Fungsi degree digunakan untuk menghitung derajat keanggotaan dari suatu nilai input. Fungsi ini mengganti variabel x dengan nilai yang dimasukkan pengguna, lalu hasilnya diubah menjadi angka desimal.

Contohnya, jika kualitas makanan adalah 7.5, fungsi ini menghitung seberapa besar nilai tersebut termasuk dalam kategori "Buruk", "Biasa", atau "Enak".

Fungsi input_range

```

In [41]: def input_range(teks, minimum, maksimum):
          while True:
              try:
                  nilai = float(input(teks))
                  if minimum <= nilai <= maksimum:
                      return nilai
                  print(f"Input harus dalam range {minimum}-{maksimum}.")
              except ValueError:
                  print("Input harus berupa angka.")

```

Fungsi input_range digunakan untuk menerima input dari pengguna sekaligus memvalidasi rentangnya. Program hanya menerima angka yang berada dalam batas minimum dan maksimum.

Pada kasus ini, kualitas makanan dan pelayanan harus berada dalam rentang 0

sampai 10. Jika input di luar rentang atau bukan angka, program akan meminta input ulang.

Fungsi kategori_tip

```
In [42]: def kategori_tip(nilai):  
        if nilai < 10:  
            return "Rendah"  
        if nilai < 15:  
            return "Sedang"  
        if nilai <= 20:  
            return "Tinggi"  
        return "Sangat Tinggi"
```

Fungsi kategori_tip digunakan untuk mengubah hasil akhir persentase tip menjadi kategori bahasa.

Jika nilai tip kurang dari 10, kategorinya "Rendah". Jika kurang dari 15, kategorinya "Sedang". Jika sampai 20, kategorinya "Tinggi". Jika lebih dari 20, kategorinya "Sangat Tinggi".

Input makanan dan pelayanan

```
In [43]: makanan_input = input_range("Masukkan kualitas makanan (0-10): ", 0  
pelayanan_input = input_range("Masukkan kualitas pelayanan (0-10): ")
```

Bagian ini meminta pengguna memasukkan kualitas makanan dan kualitas pelayanan. Keduanya dinilai dari 0 sampai 10.

Nilai ini menjadi input utama sistem fuzzy. Setelah input diterima, program akan menghitung derajat keanggotaannya pada masing-masing kategori.

Fungsi keanggotaan makanan dan pelayanan

```
In [44]: makanan_mf = {  
        "Buruk": trapmf(0, 0, 2, 4),  
        "Biasa": trimf(3, 5, 7),  
        "Enak": trapmf(6, 8, 10, 10)  
    }  
  
    pelayanan_mf = {  
        "Buruk": trapmf(0, 0, 2, 4),  
        "Biasa": trimf(3, 5, 7),  
        "Baik": trapmf(6, 8, 10, 10)  
    }
```

Bagian ini mendefinisikan kategori fuzzy untuk makanan dan pelayanan.

Kualitas makanan memiliki kategori "Buruk", "Biasa", dan "Enak". Kualitas pelayanan memiliki kategori "Buruk", "Biasa", dan "Baik".

Kategori "Buruk" dan kategori terbaik menggunakan fungsi trapesium karena berada di ujung skala. Kategori "Biasa" menggunakan fungsi segitiga karena berada di tengah skala.

Proses fuzzifikasi

```
In [45]: makanan = {k: degree(v, makanan_input) for k, v in makanan_mf.items}
pelayanan = {k: degree(v, pelayanan_input) for k, v in pelayanan_mf
```

Bagian ini menghitung derajat keanggotaan input makanan dan pelayanan terhadap masing-masing kategori.

Hasilnya adalah nilai fuzzy, misalnya makanan 7.5 memiliki derajat tertentu terhadap kategori "Enak", sedangkan pelayanan 4.0 memiliki derajat tertentu terhadap kategori "Biasa".

Nilai inilah yang digunakan untuk menentukan aturan fuzzy mana yang aktif.

Aturan fuzzy

```
In [46]: rules = [
    ("R1", "Buruk", "Buruk", 5),
    ("R2", "Buruk", "Biasa", 8),
    ("R3", "Buruk", "Baik", 12),
    ("R4", "Biasa", "Buruk", 8),
    ("R5", "Biasa", "Biasa", 15),
    ("R6", "Biasa", "Baik", 18),
    ("R7", "Enak", "Buruk", 12),
    ("R8", "Enak", "Biasa", 18),
    ("R9", "Enak", "Baik", 25)
]
```

Bagian ini berisi sembilan aturan fuzzy. Aturan dibuat dari kombinasi tiga kategori makanan dan tiga kategori pelayanan.

Setiap aturan menghasilkan nilai z, yaitu nilai persentase tip tertentu. Misalnya, jika makanan "Enak" dan pelayanan "Baik", maka nilai tip yang diberikan adalah 25%.

Karena metode yang digunakan adalah Sugeno, output setiap aturan berupa nilai tegas, bukan fungsi keanggotaan.

Evaluasi aturan

```
In [47]: evaluasi = []
for kode, m, p, z in rules:
    w = min(makanan[m], pelayanan[p])
    evaluasi.append((kode, w, z))
```

Bagian ini menghitung kekuatan setiap aturan atau w. Nilai w diperoleh dari nilai minimum antara derajat keanggotaan makanan dan pelayanan.

Operator minimum digunakan karena aturan memakai logika AND. Jika salah satu kondisi memiliki nilai kecil, maka kekuatan aturan juga menjadi kecil.

Setiap hasil aturan disimpan dalam bentuk kode aturan, nilai w, dan nilai output z.

Perhitungan defuzzifikasi Sugeno

```
In [48]: numerator = sum(w * z for _, w, z in evaluasi)
denominator = sum(w for _, w, _ in evaluasi)
tip = numerator / denominator if denominator != 0 else 0.0
```

Bagian ini menghitung hasil akhir tip menggunakan metode Sugeno.

Program menghitung numerator dari jumlah perkalian antara w dan z. Setelah itu, program menghitung denominator dari jumlah seluruh nilai w.

Nilai tip akhir diperoleh dari numerator / denominator. Jika tidak ada aturan aktif, hasil tip dibuat 0 agar program tidak mengalami pembagian dengan nol.

Aturan aktif

```
In [49]: aktif = [kode for kode, w, _ in evaluasi if w > 0]
```

Bagian ini mencari aturan mana saja yang memiliki nilai w lebih dari 0. Aturan tersebut dianggap aktif karena ikut memengaruhi hasil akhir.

Pada contoh output, aturan yang aktif adalah R8 karena makanan masuk kategori "Enak" dan pelayanan masuk kategori "Biasa".

Output

```
In [50]: print("INPUT:")
print(f"Kualitas Makanan: {makanan_input:.2f} / 10")
print(f"Kualitas Pelayanan: {pelayanan_input:.2f} / 10")
```

```
INPUT:
Kualitas Makanan: 7.50 / 10
Kualitas Pelayanan: 4.00 / 10
```

```
In [51]: print("\n-----")
print("FUZZIFIKASI")
print("-----")
print("Makanan:")
print(f"Buruk: {makanan['Buruk']:.6f}")
print(f"Biasa: {makanan['Biasa']:.6f}")
print(f"Enak: {makanan['Enak']:.6f}")
```

FUZZIFIKASI

Makanan:

Buruk: 0.000000

Biasa: 0.000000

Enak: 0.750000

```
In [52]: print("Pelayanan:")
print(f"Buruk: {pelayanan['Buruk']:.6f}")
print(f"Biasa: {pelayanan['Biasa']:.6f}")
print(f"Baik: {pelayanan['Baik']:.6f}")
```

Pelayanan:

Buruk: 0.000000

Biasa: 0.500000

Baik: 0.000000

```
In [53]: print("\n-----")
print("EVALUASI ATURAN")
print("-----")
for i in range(0, len(evaluasi), 2):
    kiri = evaluasi[i]
    if i + 1 < len(evaluasi):
        kanan = evaluasi[i + 1]
        print(f"{kiri[0]}: w={kiri[1]:.6f}, z={kiri[2]} | {kanan[0]}")
    else:
        print(f"{kiri[0]}: w={kiri[1]:.6f}, z={kiri[2]}")
```

EVALUASI ATURAN

R1: w=0.000000, z=5 | R2: w=0.000000, z=8

R3: w=0.000000, z=12 | R4: w=0.000000, z=8

R5: w=0.000000, z=15 | R6: w=0.000000, z=18

R7: w=0.000000, z=12 | R8: w=0.500000, z=18

R9: w=0.000000, z=25

```
In [54]: print("\n-----")
print("DEFUZZIFIKASI SUGENO")
print("-----")
print(f"Numerator:  $\Sigma(w_i \times z_i)$  = {numerator:.6f}")
print(f"Denominator:  $\Sigma(w_i)$  = {denominator:.6f}")
print(f"Tip = {tip:.2f}%")
print(f"Kategori: {kategori_tip(tip)}")
```

DEFUZZIFIKASI SUGENO

Numerator: $\Sigma(w_i \times z_i)$ = 9.000000

Denominator: $\Sigma(w_i)$ = 0.500000

Tip = 18.00%

Kategori: Tinggi

```
In [55]: print("\n-----")
print("TOLAK UKUR VALIDASI:")
print("-----")
```

```

print(f"μ_Enak({makanan_input:.1f}) = ({makanan_input:.1f}-6)/(8-6)
print(
    f"μ_Biasa_pelayanan({pelayanan_input:.1f}) = ({pelayanan_input:
print(
    f"Actually: pelayanan={pelayanan_input:.1f}, triangular(3,5,7):
print(f"Koreksi: μ_Biasa_pelayanan({pelayanan_input:.1f}) = {pelaya
print(
    f"R8: min({makanan['Enak']:.2f}, {pelayanan['Biasa']:.2f}) = {e
print(
    f"Juga R4: min({makanan['Biasa']:.0f}, {pelayanan['Buruk']:.0f}

print("\nRecalculate properly:")
print(f"Pelayanan={pelayanan_input:.1f}:")
print(f"Buruk: trapezoidal(0,0,2,4) - ({pelayanan_input:.0f}-4)/(4-
print(f"Biasa: triangular(3,5,7) - ({pelayanan_input:.0f}-3)/(5-3) :
print(f"Baik: trapezoidal(6,8,10,10) - {pelayanan['Baik']:.0f}")

print(f"\nMakanan={makanan_input:.1f}:")
print(f"Buruk: trapezoidal(0,0,2,4) - {makanan['Buruk']:.0f}")
print(f"Biasa: triangular(3,5,7) - {makanan_input:.1f}>=7 - {makanan
print(f"Enak: trapezoidal(6,8,10,10) - ({makanan_input:.1f}-6)/(8-6

print(f"\nActive rules: R8 = min({makanan['Enak']:.2f}, {pelayanan[
print(f"Output = ({evaluasi[7][1]:.1f}×18)/({evaluasi[7][1]:.1f}) =
print(f"Kategori: {kategori_tip(tip)} (15-20)")

```

TOLAK UKUR VALIDASI:

$\mu_{Enak}(7.5) = (7.5-6)/(8-6) = 0.75$
 $\mu_{Biasa_pelayanan}(4.0) = (4-3)/(5-3) = 0.50$ WAIT - recalculate:
Actually: pelayanan=4.0, triangular(3,5,7): $(4-3)/(5-3) = 0.50$
Koreksi: $\mu_{Biasa_pelayanan}(4.0) = 0.50$, bukan 1.0
 $R8: \min(0.75, 0.50) = 0.50$, $R5: \min(0, 0.50) = 0$
Juga $R4: \min(0, 0) = 0$, tapi pelayanan Buruk(4) = 0 karena $4 \geq 4 - 0$

Recalculate properly:

Pelayanan=4.0:

Buruk: $\text{trapezoidal}(0,0,2,4) - (4-4)/(4-2) = 0.0$ ($x \geq d - 0$)

Biasa: $\text{triangular}(3,5,7) - (4-3)/(5-3) = 0.5$

Baik: $\text{trapezoidal}(6,8,10,10) - 0$

Makanan=7.5:

Buruk: $\text{trapezoidal}(0,0,2,4) - 0$

Biasa: $\text{triangular}(3,5,7) - 7.5 \geq 7 - 0$

Enak: $\text{trapezoidal}(6,8,10,10) - (7.5-6)/(8-6) = 0.75$

Active rules: $R8 = \min(0.75, 0.5) = 0.5$

Output = $(0.5 \times 18)/(0.5) = 18.00\%$

Kategori: Tinggi (15-20)

Penjelasan Dan Output Soal 3

Import SymPy dan variabel x

```
In [20]: import sympy as sp  
  
x = sp.symbols("x")
```

Bagian ini memanggil library SymPy untuk membuat fungsi matematika simbolik. Variabel x dibuat sebagai simbol yang nantinya digunakan dalam fungsi keanggotaan fuzzy. Semua fungsi fuzzy seperti jarak, kecepatan, dan arah akan memakai x sebagai variabel dasarnya.

Fungsi trimf

```
In [21]: def trimf(a, b, c):  
    return sp.Piecewise(  
        (0, x <= a),  
        ((x - a) / (b - a), x <= b),  
        ((c - x) / (c - b), x < c),  
        (0, True)  
    )
```

Fungsi trimf digunakan untuk membuat fungsi keanggotaan berbentuk segitiga. Fungsi ini menerima tiga parameter, yaitu batas awal, titik puncak, dan batas akhir.

Nilai keanggotaan akan bernilai 0 sebelum batas awal, naik secara bertahap sampai titik puncak, lalu turun kembali sampai batas akhir. Fungsi ini cocok untuk kategori tengah seperti "Sedang", "Normal", "Lurus", "Belok_Kiri", dan "Belok_Kanan".

Fungsi trapmf

```
In [22]: def trapmf(a, b, c, d):  
    if a == b:  
        return sp.Piecewise(  
            (0, x < a),  
            (1, x <= c),  
            ((d - x) / (d - c), x < d),  
            (0, True)  
        )  
  
    if c == d:  
        return sp.Piecewise(  
            (0, x <= a),  
            ((x - a) / (b - a), x < b),
```

```

        (1, x <= d),
        (0, True)
    )

    return sp.Piecewise(
        (0, x <= a),
        ((x - a) / (b - a), x < b),
        (1, x <= c),
        ((d - x) / (d - c), x < d),
        (0, True)
    )

```

Fungsi trapmf digunakan untuk membuat fungsi keanggotaan berbentuk trapesium. Fungsi ini menerima empat parameter sebagai batas bentuk trapesium.

Jika nilai awal sama, fungsi langsung bernilai penuh pada sisi kiri. Jika nilai akhir sama, fungsi dapat tetap bernilai penuh sampai sisi kanan. Kondisi ini digunakan untuk kategori ekstrem seperti "Dekat", "Jauh", "Lambat", "Kencang", "Belok_Kiri_Tajam", dan "Belok_Kanan_Tajam".

Fungsi degree

```

In [23]: def degree(expr, nilai):
          return float(expr.subs(x, nilai))

```

Fungsi degree digunakan untuk menghitung derajat keanggotaan dari suatu nilai input. Fungsi ini mengganti simbol x dengan nilai aktual, lalu mengubah hasilnya menjadi angka desimal.

Dengan fungsi ini, program bisa mengetahui seberapa besar jarak sensor kiri atau kanan masuk ke kategori "Dekat", "Sedang", atau "Jauh".

Fungsi input_range

```

In [24]: def input_range(teks, minimum, maksimum):
          while True:
              try:
                  nilai = float(input(teks))
                  if minimum <= nilai <= maksimum:
                      return nilai
                  print(f"Input harus dalam range {minimum}-{maksimum}.")
              except ValueError:
                  print("Input harus berupa angka.")

```

Fungsi input_range digunakan untuk meminta input dari pengguna dan memastikan nilainya valid.

Program hanya menerima input angka yang berada dalam rentang minimum dan maksimum. Jika nilai di luar rentang, program menampilkan pesan

kesalahan. Jika input bukan angka, program juga menampilkan pesan bahwa input harus berupa angka.

Fungsi centroid

```
In [25]: def centroid(output_mf, aturan, start, end, step):
    total_titik = int((end - start) / step) + 1
    pembilang = 0.0
    penyebut = 0.0

    for i in range(total_titik):
        nilai = start + i * step
        mu = max(min(w, degree(output_mf[label], nilai)) for label,
        pembilang += nilai * mu
        penyebut += mu

    return pembilang / penyebut if penyebut != 0 else 0.0
```

Fungsi centroid digunakan untuk melakukan defuzzifikasi dengan metode centroid.

Fungsi ini menghitung nilai akhir dari output fuzzy berdasarkan aturan yang aktif. Program membagi rentang output menjadi banyak titik sesuai nilai step, lalu menghitung nilai keanggotaan pada setiap titik.

Nilai $\min(w, \text{degree}(\dots))$ digunakan untuk membatasi output berdasarkan firing strength aturan. Nilai $\max(\dots)$ digunakan untuk mengambil gabungan terbesar dari semua aturan yang menghasilkan output. Hasil akhirnya diperoleh dari pembilang dibagi penyebut.

Jika penyebut bernilai 0, maka fungsi mengembalikan nilai 0 agar tidak terjadi pembagian dengan nol.

Fungsi interpretasi_arah

```
In [26]: def interpretasi_arah(nilai):
    if nilai < -45:
        return "Kiri Tajam"
    if nilai < -10:
        return "Kiri"
    if nilai <= 10:
        return "Lurus"
    if nilai <= 45:
        return "Kanan"
    return "Kanan Tajam"
```

Fungsi interpretasi_arah digunakan untuk mengubah nilai sudut arah menjadi kategori belokan.

Jika nilai sangat negatif, robot dianggap belok kiri tajam. Jika negatif sedang,

robot belok kiri. Jika berada di sekitar nol, robot berjalan lurus. Jika positif, robot belok kanan. Jika sangat positif, robot belok kanan tajam.

Fungsi interpretasi_kecepatan

```
In [27]: def interpretasi_kecepatan(nilai):  
        if nilai <= 35:  
            return "Lambat"  
        if nilai <= 70:  
            return "Normal"  
        return "Kencang"
```

Fungsi interpretasi_kecepatan digunakan untuk mengubah nilai kecepatan menjadi kategori bahasa.

Jika nilai kecepatan sampai 35, hasilnya "Lambat". Jika sampai 70, hasilnya "Normal". Jika lebih dari 70, hasilnya "Kencang".

Input jarak sensor kiri dan kanan

```
In [28]: kiri_input = input_range("Masukkan jarak sensor kiri (0-200): ", 0,  
kanan_input = input_range("Masukkan jarak sensor kanan (0-200): ", 0,
```

Bagian ini meminta pengguna memasukkan jarak sensor kiri dan sensor kanan.

Kedua input dibatasi dari 0 sampai 200 cm. Input ini menjadi data utama untuk menentukan arah belok dan kecepatan robot.

Fungsi keanggotaan jarak, kecepatan, arah, dan fuzzifikasi input

```
In [29]: jarak_mf = {  
        "Dekat": trapmf(0, 0, 20, 60),  
        "Sedang": trimf(40, 100, 160),  
        "Jauh": trapmf(140, 180, 200, 200)  
    }  
  
    kecepatan_mf = {  
        "Lambat": trapmf(0, 0, 15, 35),  
        "Normal": trimf(25, 50, 75),  
        "Kencang": trapmf(65, 85, 100, 100)  
    }  
  
    arah_mf = {  
        "Belok_Kiri_Tajam": trapmf(-90, -90, -70, -30),  
        "Belok_Kiri": trimf(-50, -25, 0),  
        "Lurus": trimf(-15, 0, 15),  
        "Belok_Kanan": trimf(0, 25, 50),  
        "Belok_Kanan_Tajam": trapmf(30, 70, 90, 90)  
    }
```

```
kiri = {k: degree(v, kiri_input) for k, v in jarak_mf.items()}
kanan = {k: degree(v, kanan_input) for k, v in jarak_mf.items()}
```

Bagian ini mendefinisikan tiga kelompok fungsi keanggotaan fuzzy.

Jarak sensor memiliki kategori "Dekat", "Sedang", dan "Jauh". Kecepatan memiliki kategori "Lambat", "Normal", dan "Kencang". Arah belok memiliki kategori "Belok_Kiri_Tajam", "Belok_Kiri", "Lurus", "Belok_Kanan", dan "Belok_Kanan_Tajam".

Setelah semua fungsi keanggotaan dibuat, program langsung menghitung derajat keanggotaan untuk sensor kiri dan kanan. Hasilnya disimpan dalam variabel kiri dan kanan.

Aturan fuzzy

```
In [30]: rules = [
    ("R1", "Dekat", "Dekat", "Lambat", "Lurus"),
    ("R2", "Dekat", "Sedang", "Lambat", "Belok_Kanan"),
    ("R3", "Dekat", "Jauh", "Normal", "Belok_Kanan_Tajam"),
    ("R4", "Sedang", "Dekat", "Lambat", "Belok_Kiri"),
    ("R5", "Sedang", "Sedang", "Normal", "Lurus"),
    ("R6", "Sedang", "Jauh", "Normal", "Belok_Kanan"),
    ("R7", "Jauh", "Dekat", "Normal", "Belok_Kiri_Tajam"),
    ("R8", "Jauh", "Sedang", "Normal", "Belok_Kiri"),
    ("R9", "Jauh", "Jauh", "Kencang", "Lurus")
]
```

Bagian ini berisi sembilan aturan fuzzy.

Setiap aturan terdiri dari kode aturan, kondisi sensor kiri, kondisi sensor kanan, output kecepatan, dan output arah belok.

Contohnya, jika sensor kiri "Dekat" dan sensor kanan "Jauh", robot bergerak dengan kecepatan "Normal" dan arah "Belok_Kanan_Tajam". Aturan ini menunjukkan bahwa robot harus menjauh dari sisi kiri karena objek di kiri lebih dekat.

Firing strength, defuzzifikasi kecepatan dan arah, serta aturan aktif

```
In [31]: firing = []
for kode, k, r, output_kecepatan, output_arah in rules:
    w = min(kiri[k], kanan[r])
    firing.append((kode, output_kecepatan, output_arah, w))

kecepatan = centroid(kecepatan_mf, [(ok, w) for _, ok, _, w in firing])
arah = centroid(arah_mf, [(oa, w) for _, _, oa, w in firing], -90, 90)
aktif = [kode for kode, _, _, w in firing if w > 0]
```

Bagian ini menghitung kekuatan aktif setiap aturan fuzzy.

Nilai w dihitung menggunakan nilai minimum antara derajat keanggotaan sensor kiri dan sensor kanan. Nilai minimum digunakan karena aturan memakai operator AND.

Setiap hasil aturan disimpan ke dalam firing, berisi kode aturan, output kecepatan, output arah, dan nilai firing strength.

Setelah itu, program melakukan defuzzifikasi untuk dua output, yaitu kecepatan dan arah. Kecepatan dihitung pada rentang 0 sampai 100 dengan step 0.5. Arah dihitung pada rentang -90 sampai 90 dengan step 0.5.

Terakhir, program membuat daftar aktif, yaitu aturan yang memiliki nilai firing strength lebih dari 0.

Output hasil

```
In [32]: print("INPUT:")
print(f"Jarak Kiri: {kiri_input:.2f} cm")
print(f"Jarak Kanan: {kanan_input:.2f} cm")
```

```
INPUT:
Jarak Kiri: 30.00 cm
Jarak Kanan: 150.00 cm
```

```
In [33]: print("\n-----")
print("FUZZIFIKASI")
print("-----")
print(f"Sensor Kiri ({kiri_input:.0f} cm):")
print(f"Dekat: {kiri['Dekat']:.6f}")
print(f"Sedang: {kiri['Sedang']:.6f}")
print(f"Jauh: {kiri['Jauh']:.6f}")
```

```
-----
FUZZIFIKASI
-----
```

```
Sensor Kiri (30 cm):
Dekat: 0.750000
Sedang: 0.000000
Jauh: 0.000000
```

```
In [34]: print(f"Sensor Kanan ({kanan_input:.0f} cm):")
print(f"Dekat: {kanan['Dekat']:.6f}")
print(f"Sedang: {kanan['Sedang']:.6f}")
print(f"Jauh: {kanan['Jauh']:.6f}")
```

```
Sensor Kanan (150 cm):
Dekat: 0.000000
Sedang: 0.166667
Jauh: 0.250000
```

```
In [35]: print("\n-----")
print("FIRING STRENGTH")
```

```
print("-----")
for i in range(0, len(firing), 3):
    baris = firing[i:i + 3]
    print(" ".join(f"{kode}: {w:.6f}" for kode, _, _, w in baris))
```

FIRING STRENGTH

R1: 0.000000 R2: 0.166667 R3: 0.250000
R4: 0.000000 R5: 0.000000 R6: 0.000000
R7: 0.000000 R8: 0.000000 R9: 0.000000

```
In [36]: print("\n-----")
print("DEFUZZIFIKASI (Centroid)")
print("-----")
print(f"Kecepatan: {kecepatan:.2f} cm/s")
print(f"Arah Belok: {arah:.2f} derajat")
print(f"Interpretasi: Belok {interpretasi_arah(arah)} dengan kecepatan
```

DEFUZZIFIKASI (Centroid)

Kecepatan: 39.25 cm/s
Arah Belok: 50.28 derajat
Interpretasi: Belok Kanan Tajam dengan kecepatan Normal

```
In [37]: print("\n-----")
print("TOLAK UKUR VALIDASI")
print("-----")
print(f"μ_Dekat_kiri({kiri_input:.2f}) = {kiri['Dekat']:.6f}")
print(f"μ_Sedang_kanan({kanan_input:.2f}) = {kanan['Sedang']:.6f}")
print(f"μ_Jauh_kanan({kanan_input:.2f}) = {kanan['Jauh']:.6f}")
print(f"Aturan aktif: {'', '.join(aktif) if aktif else 'Tidak ada'}")
print(f"Robot belok kanan karena arah bernilai positif: {'Ya' if ar")
print(f"Kecepatan moderate antara lambat dan normal: {'Ya' if 25 <=
```

TOLAK UKUR VALIDASI

μ_Dekat_kiri(30.00) = 0.750000
μ_Sedang_kanan(150.00) = 0.166667
μ_Jauh_kanan(150.00) = 0.250000
Aturan aktif: R2, R3
Robot belok kanan karena arah bernilai positif: Ya
Kecepatan moderate antara lambat dan normal: Ya